Autor: Iván García Santillán

Algoritmos predictivos de clasificación: redes neuronales artificiales

Introducción

Una red neuronal artificial (RNA), o artificial neural network (ANN), es un modelo de aprendizaje automático inspirado en el funcionamiento del cerebro humano. Está compuesta por unidades de procesamiento llamadas neuronas artificiales o nodos, que están organizadas en capas interconectadas.

Las redes neuronales artificiales son utilizadas para abordar una amplia variedad de tareas de aprendizaje automático, incluyendo clasificación, regresión, procesamiento de imágenes, procesamiento de lenguaje natural y más.

Por ejemplo, en el campo de la visión por computadora, son esenciales para el reconocimiento de imágenes y rostros, permitiendo innovaciones como los sistemas de seguridad biométrica y las aplicaciones de filtros en redes sociales. En el procesamiento del lenguaje natural, impulsan traductores automáticos y asistentes virtuales, facilitando la comunicación y la interacción con la tecnología. En el sector financiero, estas redes se utilizan para la predicción de series temporales en mercados bursátiles y para la detección de fraudes, analizando patrones complejos en transacciones.

En medicina, contribuyen significativamente en el diagnóstico y pronóstico de enfermedades, analizando datos médicos y patrones en imágenes diagnósticas como radiografías y resonancias magnéticas. Además, en el transporte y ámbito del vehículo autónomo, son fundamentales para la interpretación de los datos sensoriales y la toma de decisiones en tiempo real.

Aunque también se debe considerar algunas desventajas del algoritmo como la baja velocidad de entrenamiento, uso de memoria medio o alto, considerable afinamiento requerido y la baja interpretabilidad de sus resultados ya que son considerados como cajas negras. A pesar de esto, las RNA son muy versátiles

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

para cualquier tipo de aplicación con buenos resultados. Por lo que su uso es muy extendido en el mundo de la ciencia de datos e IA, considerando que son la base de otros algoritmos más complejos de Deep Learning.

A continuación, se detallan el funcionamiento del algoritmo y un ejemplo en Python, Keras y TensorFlow para la predicción del cáncer de mama utilizando un data set público.

Contenido

Una red neuronal artificial es un modelo de aprendizaje automático inspirado en el funcionamiento del cerebro humano. Algunos conceptos clave relacionados con las redes neuronales artificiales son los siguientes (Berzal, 2019):

Neuronas Artificiales: cada neurona artificial es una unidad de procesamiento que toma una serie de entradas, las procesa y produce una salida. Las entradas se multiplican por pesos, se suman y se pasa el resultado a una función de activación.

Función de activación: introduce no linealidad en la red, permitiendo que las redes neuronales aprendan relaciones y patrones complejos en los datos. Existen varias funciones de activación comunes utilizadas en las redes neuronales, entre las que se incluyen: Sigmoide, ReLU, Tangente Hiperbólica, etc. La elección de la función de activación depende del tipo de problema y la arquitectura de la RNA. La función de activación juega un papel fundamental en la capacidad de la RNA para aprender y generalizar a partir de los datos de entrenamiento.

Capas: las neuronas se organizan en capas. Una red neuronal típica consta de una capa de entrada, una o más capas ocultas y una capa de salida. Las capas ocultas permiten a la red aprender representaciones intermedias y abstraer características de los datos.

Conexiones: cada neurona en una capa está conectada a todas las neuronas de la capa anterior y a todas las de la capa siguiente. Estas conexiones tienen pesos que se ajustan (aprenden) durante el proceso de entrenamiento.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

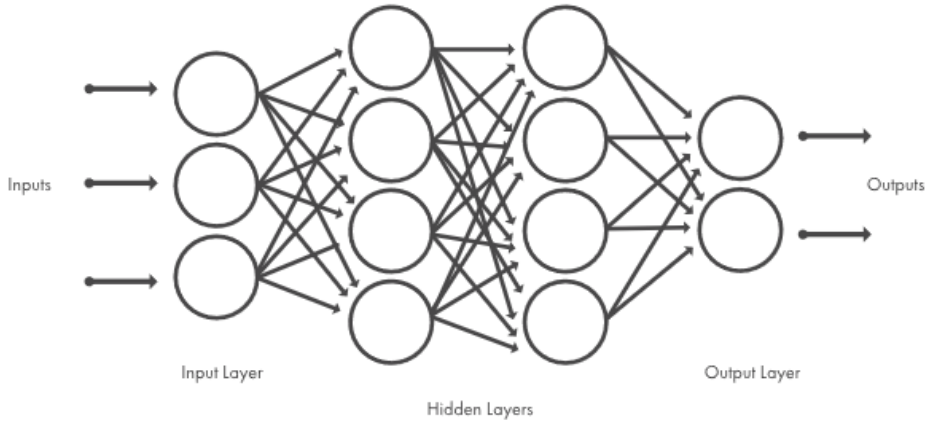
Entrenamiento: el entrenamiento de una red neuronal implica el ajuste de los pesos de las conexiones para que la red pueda hacer predicciones precisas. Esto se hace utilizando algoritmos de optimización y un conjunto de datos de entrenamiento que contiene ejemplos de entrada y las salidas deseadas (etiquetas).

Función de Costo: la función de costo mide la discrepancia (error) entre las predicciones de la red y las salidas reales (ground truth) en el conjunto de entrenamiento. El objetivo del entrenamiento es minimizar esta función de costo.

Backpropagation: es un algoritmo utilizado para propagar el error desde la capa de salida hacia atrás a través de la red, ajustando los pesos de las conexiones en función del error cometido.

Arquitectura: las redes neuronales pueden tener diferentes arquitecturas, como redes neuronales feed-forward (Figura 23), redes neuronales recurrentes (RNN) y redes neuronales convolucionales (CNN), cada una adaptada a tareas específicas. En este curso veremos las primeras, conocida como MLP (multi-layer perceptron).

Figura 23. Arquitectura típica de una red neuronal artificial

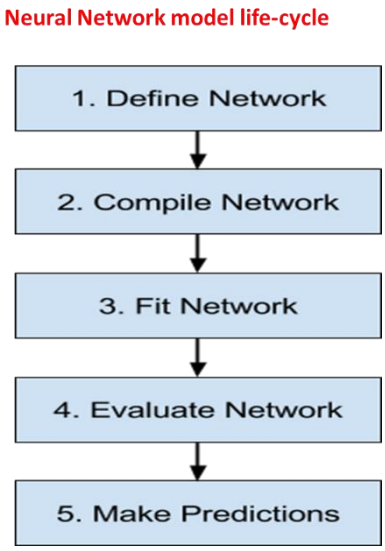


Las redes neuronales artificiales se han destacado en una amplia gama de aplicaciones, incluyendo el procesamiento de imágenes, el procesamiento de lenguaje natural, la visión por computadora, la traducción automática, el reconocimiento de voz, la recomendación de contenido y muchas otras. Su capacidad para aprender representaciones complejas y realizar tareas

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

sofisticadas las ha convertido en un componente esencial del campo de la inteligencia artificial y el aprendizaje automático. En general, el ciclo de vida de un modelo de red neuronal artificial se muestra en la Figura 24.

Figura 24. Ciclo de vida de un modelo de red neuronal artificial



A continuación, se presenta un ejemplo del algoritmo RNA en Python, Keras y TensorFlow para la predicción del cáncer de mama (benigno=0, maligno=1) utilizando el conjunto de datos Breast Cancer Wisconsin (Diagnostic) disponible en la librería scikit-learn.

```
# RNA for breast cancer

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import classification_report
from keras.models import Sequential
from keras.layers.core import Dense

# Carga el conjunto de datos Breast Cancer
dataset = load_breast_cancer()
X = dataset.data # 569x30
y = dataset.target # 569x1
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

```
# Divide el conjunto de datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Normaliza los datos para que tengan una escala similar
scaler = MinMaxScaler(feature_range=(0,1)) # [0, 1]
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Crea y entrena el modelo RNA
model = Sequential()
model.add(Dense(10, activation='relu', input_dim=30))
model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])

model.fit(X_train, y_train, epochs=100)

# Realiza predicciones usando el conjunto de prueba
y_pred = model.predict(X_test)

# Convierte las probabilidades en etiquetas binarias (0 o 1)
y_pred = (y_pred > 0.5)

# Muestra el informe de evaluación del modelo entrenado
print(classification_report(y_test, y_pred))

# Matriz de confusión:
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)
print("confusion matrix: \n", cm)
# gráfica de la CM
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

```
plt.figure(figsize = (8,4))
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel('Prediction', fontsize = 12)
plt.ylabel('Real', fontsize = 12)
plt.show()

# Exactitud:
from sklearn.metrics import accuracy_score
acc = accuracy_score(y_test, y_pred)
print("accuracy: ", acc)

# Sensibilidad:
from sklearn.metrics import recall_score
recall = recall_score(y_test, y_pred)
print("recall: ", recall)

# Precisión:
from sklearn.metrics import precision_score
precision = precision_score(y_test, y_pred)
print("precision: ", precision)

# Especificidad
# 'specificity' is just a special case of 'recall'.
# specificity is the recall of the negative class
specificity = recall_score(y_test, y_pred, pos_label=0)
print("specificity: ", specificity)

# Puntuación F1:
from sklearn.metrics import f1_score
f1 = f1_score(y_test, y_pred)
print("f1 score: ", f1)

# Área bajo la curva:
from sklearn.metrics import roc_auc_score
auc = roc_auc_score(y_test, y_pred)
print("auc: ", auc)
# Curva ROC
```

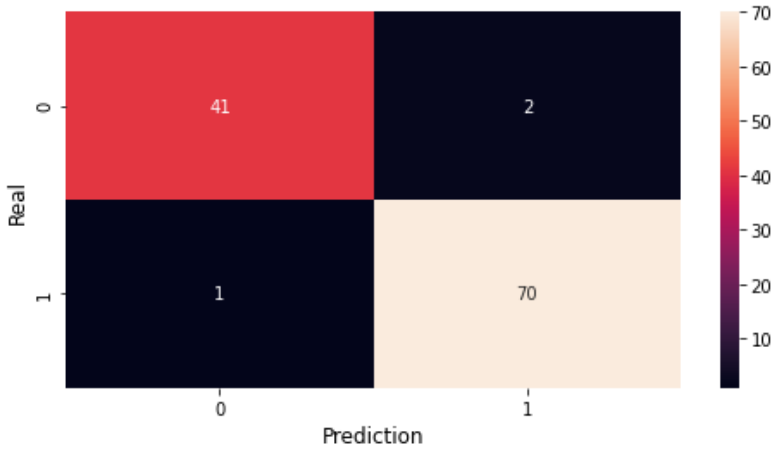
Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

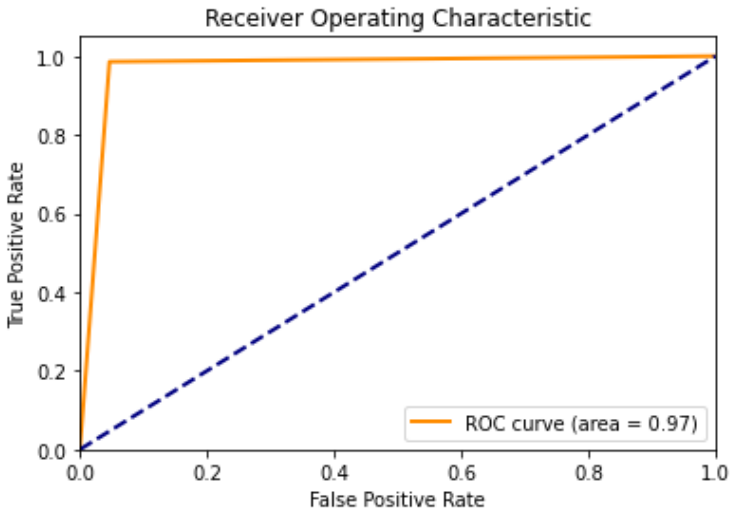
```
from sklearn.metrics import roc_curve
plt.figure()
lw = 2
plt.plot(roc_curve(y_test, y_pred)[0], roc_curve(y_test, y_pred)[1],
color='darkorange',lw=lw, label='ROC curve (area = %0.2f)' %auc)
plt.plot([0, 1], [0, 1], color='navy', lw=lw, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc="lower right")
plt.show()

# R Score (R^2 coefficient of determination)
from sklearn.metrics import r2_score
R = r2_score(y_test, y_pred)
print("R2: ", R)
```

Los resultados del modelo de RNA se presentan a continuación:

accuracy: 0.97
recall: 0.99
precision: 0.97
specificity: 0.95
f1 score: 0.98
auc: 0.97
R2: 0.89

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.





Un clasificador adecuado es aquel que alcanza una exactitud (accuracy) superior al 85 %. Por lo que, este modelo de RNA entrenado cumple con este criterio de calidad.

Adicionalmente, tenga en cuenta que se pueden ajustar varios parámetros del algoritmo (como activation, loss, optimizer, metrics, epochs, etc.) para obtener un modelo óptimo. Revise la documentación (ayuda en línea) del algoritmo para más detalles y experimente con ellos.

Finalmente, aunque la RNA es un modelo fundamental en la caja de herramientas de análisis predictivo, como cualquier otra técnica, presenta tanto ventajas como desventajas que se resumen a continuación:

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Ventajas de las redes neuronales artificiales:

Capacidad de aprendizaje y adaptación: las RNA tienen la capacidad de aprender y modelar relaciones no lineales y complejas, lo que las hace extremadamente potentes para modelar problemas que son difíciles de resolver con técnicas tradicionales. El término "modelar relaciones no lineales" se refiere a la capacidad de un modelo estadístico o de aprendizaje automático para capturar y representar relaciones entre variables que no son simplemente proporcionales o directas. En una relación no lineal, los cambios en una variable no producen cambios proporcionales y predecibles en otra variable, lo que significa que la relación entre variables no puede ser descrita adecuadamente por una línea recta, sino por otro tipo de relación como cuadrática, exponencial, logarítmica, sinusoidal, etc.

Generalización: una vez entrenadas, las redes neuronales son capaces de generalizar a partir de ejemplos no vistos, lo que es crucial para aplicaciones como el reconocimiento de patrones y la clasificación de imágenes.

Manejo de grandes volúmenes de datos: las RNA son eficaces para manejar grandes cantidades de datos y son capaces de identificar patrones y tendencias que pueden no ser evidentes para los humanos u otros algoritmos.

Flexibilidad: pueden ser aplicadas a una amplia gama de industrias y problemas, desde el reconocimiento de voz y facial hasta la predicción de series temporales y análisis de sentimientos.

Soporte para diversos tipos de datos: capaces de procesar datos en diferentes formas, incluyendo imágenes, audio, texto y más, haciendo posible su uso en una variedad de aplicaciones en el mundo real.

Desventajas de las Redes Neuronales Artificiales:

Requieren gran cantidad de datos para el entrenamiento: para funcionar bien, las RNA generalmente requieren grandes conjuntos de datos de entrenamiento, lo que puede ser un desafío en términos de recopilación y procesamiento de datos.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Costosas computacionalmente: el entrenamiento de modelos de redes neuronales puede ser muy demandante en términos de recursos computacionales, especialmente para redes profundas con muchas capas (Deep Learning). Esto puede requerir hardware especializado, como GPUs.

Difíciles de interpretar: a diferencia de otros modelos predictivos (como los árboles de decisión), las RNA son a menudo consideradas como "cajas negras" porque las operaciones internas y la manera en que llegan a una decisión no son fácilmente interpretables.

Sobreajuste: las redes neuronales son propensas al sobreajuste, especialmente si no tienen suficientes datos de entrenamiento o si la arquitectura de la red es demasiado compleja en relación con la simplicidad del problema.

Sensibilidad a la configuración de los parámetros: la configuración de parámetros (como la tasa de aprendizaje, el número de capas, y el número de nodos por capa) puede tener un gran impacto en el rendimiento del modelo, y encontrar la configuración óptima puede ser un proceso laborioso y técnico.

Vulnerabilidad a ataques adversarios: las redes neuronales pueden ser susceptibles a ataques adversarios, en los cuales pequeñas pero intencionadas perturbaciones en los datos de entrada pueden engañar a la red para que haga predicciones incorrectas.

Conclusiones

En esta lectura se ha revisado los fundamentos del algoritmo de red neuronal artificial para abordar problemas de clasificación binaria utilizando Python y TensorFlow-Keras. Las redes neuronales ofrecen un poderoso conjunto de capacidades para el modelado de datos complejos y no lineales, pero también presentan desafíos significativos en términos de necesidades de datos, costos computacionales, y dificultades en la interpretación de sus decisiones. La elección de utilizar redes neuronales debe considerar estos factores en función del problema específico y los recursos disponibles.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Por lo que, se motiva a los estudiantes a revisar material y ejemplos adicionales disponibles libremente en Internet para profundizar en la comprensión del algoritmo y su aplicación eficaz en diferentes problemas de variada complejidad.

Bibliografía

Berzal, Fernando (2019). *Redes neuronales & Deep Learning*. USA: Independently published.

Gerón, A. (2023). *Aprende Machine Learning con Scikit-Learn, Keras y TensorFlow: Conceptos, herramientas y técnicas para conseguir sistemas inteligentes.* Tercera Edición. Anaya Multimedia.

Torres, J. (2020). *Python Deep Learning: Introducción práctica con Keras y TensorFlow 2*. Marcombo.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Autor: Iván García Santillán

Algoritmos predictivos de clasificación: Árbol de decisión

Introducción

Es un método de aprendizaje automático utilizado para la toma de decisiones (o hacer predicciones) y la resolución de problemas de clasificación y regresión. Los árboles de decisión tienen una amplia gama de aplicaciones prácticas.

En el sector financiero, se utilizan para evaluar el riesgo de crédito de los clientes y en el *trading* para tomar decisiones de inversión basadas en patrones históricos. En el ámbito de la salud, ayudan a diagnosticar enfermedades al clasificar los síntomas y signos clínicos.

En el comercio minorista y *marketing* son útiles para segmentar clientes y predecir su comportamiento de compra, lo que facilita la personalización de estrategias de *marketing*. También desempeñan un papel crucial en la calidad del control en la fabricación, identificando factores clave que influyen en los defectos del producto. Su popularidad se debe a su facilidad de interpretación y efectividad, ya que los árboles de decisión pueden manejar tanto datos numéricos como categóricos y son capaces de modelar interacciones complejas entre variables. Aunque también se debe considerar algunas desventajas como que pueden ser propensos al sobreajuste (*overfitting*) y una baja precisión en ciertos conjuntos de datos.

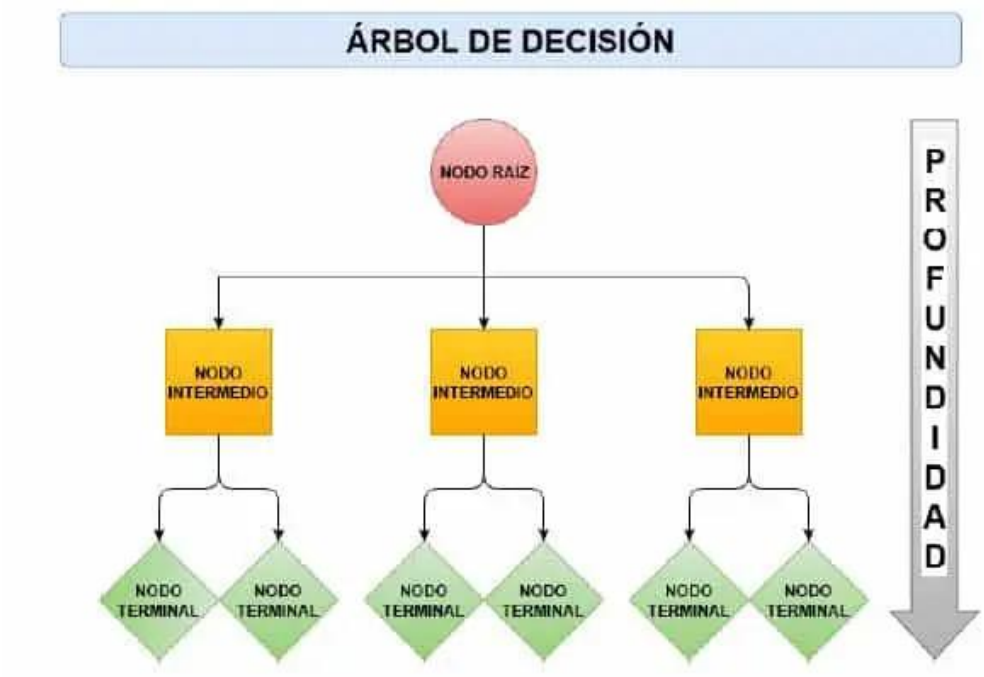
A continuación, se detallan el funcionamiento del algoritmo y un ejemplo en Python, Keras y TensorFlow para la predicción del cáncer de mama utilizando un data set público.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Contenido

El algoritmo de árbol de decisión se basa en la estructura de un árbol, donde cada nodo representa una decisión basada en una característica específica, y las ramas representan las posibles salidas o resultados de esa decisión como se indica en la Figura 21.

Figura 21. Estructura básica de un Árbol de decisión



Fuente: MaximaFormacion (2023)

Algunos conceptos clave relacionados con el algoritmo de árbol de decisión son los siguientes:

- **Nodo Raíz:** el nodo superior del árbol, que representa la característica que mejor divide el conjunto de datos inicial en función de ciertos criterios (como la ganancia de información o la impureza).

- **Nodos Internos (intermedio):** los nodos en el medio del árbol que representan decisiones basadas en características específicas.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

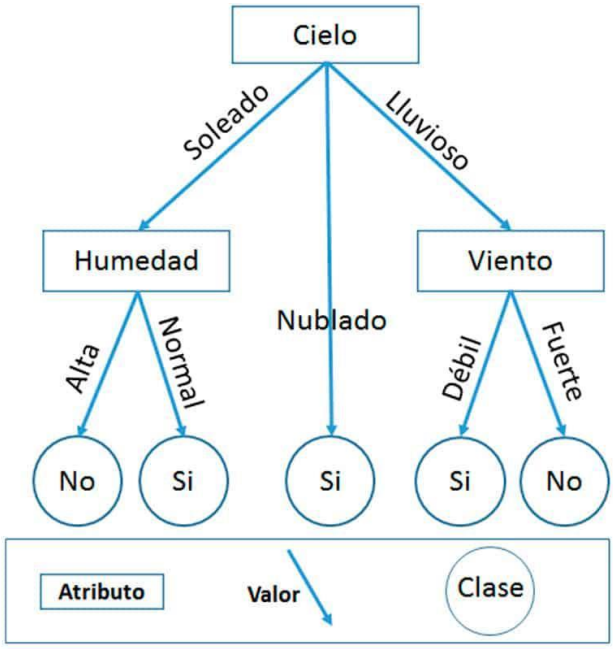
- **Hojas (nodo terminal):** los nodos finales del árbol que representan las etiquetas de clasificación o los valores de regresión resultantes.

- **División:** el proceso de dividir un nodo en dos o más nodos hijos según una característica específica. La elección de la característica y el criterio de división es un aspecto importante del algoritmo.

- **Criterios de Decisión:** los criterios utilizados para decidir cómo dividir los nodos, como la ganancia de información (entropía), la impureza Gini o el error cuadrado medio, dependiendo del tipo de problema (clasificación o regresión).

Un ejemplo de árbol de decisión para predecir si jugar o no un partido de tenis considerando variables atmosféricas se presenta en la Figura 22.

Figura 22. Árbol de decisión para predecir si jugar o no un partido de tenis



Fuente: InstitutoEmprende (2023)

Los árboles de decisión son populares en el aprendizaje automático debido a su simplicidad y capacidad para manejar conjuntos de datos tanto categóricos como numéricos. Además, son fácilmente interpretables y explicables, lo que los hace útiles en aplicaciones donde la transparencia del modelo es importante.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

A continuación, se presenta un ejemplo del algoritmo de árbol de decisión en Python, Keras y TensorFlow para la predicción del cáncer de mama (benigno=0, maligno=1) utilizando el conjunto de datos Breast Cancer Wisconsin (Diagnostic) disponible en la librería scikit-learn.

```
# Decision Tree for breast cancer

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import classification_report
from sklearn.tree import DecisionTreeClassifier

# Carga el conjunto de datos Breast Cancer
dataset = load_breast_cancer()
X = dataset.data # 569x30
y = dataset.target # 569x1

# Divide el conjunto de datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Normaliza los datos para que tengan una escala similar
scaler = MinMaxScaler(feature_range=(0,1)) # [0, 1]
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Crea y entrena el modelo de árbol de decisión
model = DecisionTreeClassifier(max_depth=4, criterion = 'gini')
model.fit(X_train, y_train)

# Realiza predicciones usando el conjunto de prueba
y_pred = model.predict(X_test)

# Convierte las probabilidades en etiquetas binarias (0 o 1)
y_pred = (y_pred > 0.5)
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

```
# Muestra el informe de evaluación del modelo entrenado
print(classification_report(y_test, y_pred))

# Matriz de confusión:
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)
print ("confusion matrix: \n", cm)
# gráfica de la CM
plt.figure(figsize = (8,4))
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel('Prediction', fontsize = 12)
plt.ylabel('Real', fontsize = 12)
plt.show()

# Exactitud:
from sklearn.metrics import accuracy_score
acc = accuracy_score(y_test, y_pred)
print("accuracy: ", acc)

# Sensibilidad:
from sklearn.metrics import recall_score
recall = recall_score(y_test, y_pred)
print("recall: ", recall)

# Precisión:
from sklearn.metrics import precision_score
precision = precision_score(y_test, y_pred)
print("precision: ", precision)

# Especificidad
# 'specificity' is just a special case of 'recall'.
# specificity is the recall of the negative class
specificity = recall_score(y_test, y_pred, pos_label=0)
print("specificity: ", specificity)
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

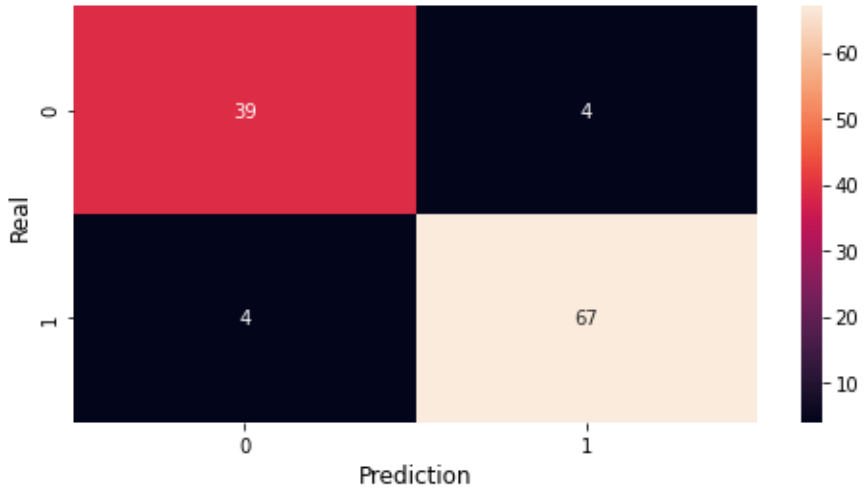
```
# Puntuación F1:
from sklearn.metrics import f1_score
f1 = f1_score(y_test, y_pred)
print("f1 score: ", f1)

# Área bajo la curva:
from sklearn.metrics import roc_auc_score
auc = roc_auc_score(y_test, y_pred)
print("auc: ", auc)

# Curva ROC
from sklearn.metrics import roc_curve
plt.figure()
lw = 2
plt.plot(roc_curve(y_test, y_pred)[0], roc_curve(y_test, y_pred)[1],
color='darkorange',lw=lw, label='ROC curve (area = %0.2f)' %auc)
plt.plot([0, 1], [0, 1], color='navy', lw=lw, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc="lower right")
plt.show()

# R Score (R^2 coefficient of determination)
from sklearn.metrics import r2_score
R = r2_score(y_test, y_pred)
print("R2: ", R)
```

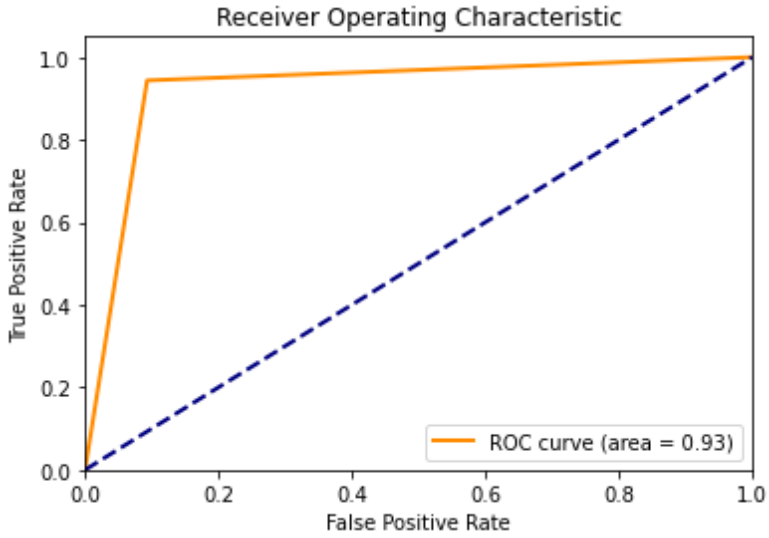
Los resultados del modelo de árbol de decisión se presentan a continuación:

```
accuracy:  0.93
recall:  0.94
precision:  0.94
specificity:  0.90
f1 score:  0.94
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

auc: 0.93
R2: 0.70





Un clasificador adecuado es aquel que alcanza una exactitud (accuracy) superior al 85 %. Por lo que, este modelo de árbol de decisión entrenado cumple con este criterio de calidad.

Adicionalmente, tenga en cuenta que se pueden ajustar varios parámetros del algoritmo (como max_depth, criterion, etc.) para obtener un modelo óptimo.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Revise la documentación (ayuda en línea https://scikit-learn.org/stable/supervised_learning.html) del algoritmo para más detalles y experimente con ellos.

Finalmente, aunque un árbol de decisión es un modelo fundamental en la caja de herramientas de análisis predictivo, como cualquier otra técnica, presenta tanto ventajas como desventajas que se resumen a continuación:

Ventajas de los árboles de decisión:

Fácil interpretación y visualización: los árboles de decisión son fáciles de entender y explicar, incluso para usuarios sin un profundo conocimiento técnico. Su representación gráfica, que muestra claramente cómo se toman las decisiones, es intuitiva y útil para la toma de decisiones basada en datos.

Manejo de datos no lineales: pueden capturar relaciones no lineales sin necesidad de transformar las variables, lo que los hace aplicables a una amplia variedad de datos. El término "modelar relaciones no lineales" se refiere a la capacidad de un modelo estadístico o de aprendizaje automático para capturar y representar relaciones entre variables que no son simplemente proporcionales o directas. En una relación no lineal, los cambios en una variable no producen cambios proporcionales y predecibles en otra variable, lo que significa que la relación entre variables no puede ser descrita adecuadamente por una línea recta, sino por otro tipo de relación como cuadrática, exponencial, logarítmica, sinusoidal, etc.

No requieren escalado de características: a diferencia de otros algoritmos que requieren normalización o escalado de datos, los árboles de decisión funcionan bien con características en su escala original.

Pueden manejar tanto variables numéricas como categóricas: los árboles de decisión pueden manejar datos que incluyan tanto variables numéricas como categóricas sin la necesidad de crear variables ficticias o dummies.

Robustos a outliers: los árboles de decisión son bastante robustos a la presencia de outliers, ya que las divisiones se hacen basadas en porcentajes de muestras y no en valores específicos.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Desventajas de los Árboles de Decisión:

Sobreajuste: una de las mayores desventajas de los árboles de decisión es su tendencia al sobreajuste, especialmente con árboles muy profundos. Los árboles que son demasiado complejos tienden a aprender detalles del conjunto de datos de entrenamiento que no generalizan bien a nuevos datos.

Inestabilidad: pequeñas variaciones en los datos pueden resultar en la generación de árboles completamente diferentes. Esta inestabilidad se puede mitigar en parte utilizando ensambles de árboles de decisión, como Random Forests o Gradient Boosting.

Problemas con datos desbalanceados: al igual que otros algoritmos, los árboles de decisión pueden crear sesgos si las clases están muy desbalanceadas.

Dificultad con datos de alta dimensionalidad: aunque pueden manejar muchas características, su rendimiento puede degradarse si muchas de estas características son irrelevantes o redundantes.

Conclusiones

En esta lectura se ha revisado los fundamentos del algoritmo de árbol de decisión para abordar problemas de clasificación binaria utilizando Python y TensorFlow-Keras. Los árboles de decisión ofrecen una metodología comprensible y versátil para abordar problemas de clasificación y regresión, pero su uso debe ser cuidadoso para evitar problemas como el sobreajuste y la inestabilidad. La utilización de técnicas de poda de árboles, la limitación de la profundidad del árbol y el uso de métodos de ensamble son estrategias comunes para mejorar el rendimiento de los árboles de decisión en la práctica.

Por lo que, se motiva a los estudiantes a revisar material y ejemplos adicionales disponibles libremente en Internet para profundizar en la comprensión del algoritmo y su aplicación eficaz en diferentes problemas de variada complejidad.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Bibliografía

Berzal, Fernando (2019). *Redes neuronales & Deep Learning*. USA: Independently published.

Gerón, A. (2023). *Aprende Machine Learning con Scikit-Learn, Keras y TensorFlow: Conceptos, herramientas y técnicas para conseguir sistemas inteligentes.* Tercera Edición. Anaya Multimedia.

Torres, J. (2020). *Python Deep Learning: Introducción práctica con Keras y TensorFlow 2*. Marcombo.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Autor:
Iván García Santillán

Algoritmos predictivos de clasificación y evaluación de modelos

Introducción

En esta unidad se utiliza herramientas y conjuntos de datos públicos para el entrenamiento de algunos algoritmos de aprendizaje automático supervisado (clasificación) siguiendo una metodología para el desarrollo de proyectos de análisis de datos. Los algoritmos de clasificación que abordaremos son árboles de decisión y redes neuronales artificiales para problemas de clasificación binaria y/o multi-clasificación, todos ellos utilizando el lenguaje Python y el framework TensorFlow-Keras. Además, se enfatizará en la evaluación de los algoritmos utilizando métricas y gráficas de rendimiento de los modelos entrenados.

Palabras clave

Aprendizaje automático, árboles de decisión, redes neuronales artificiales, evaluación de modelos.

Reto

¿Qué métricas y gráficas resultan más adecuados para evaluar el rendimiento de algoritmos de clasificación binaria/multiclase?

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Desarrollo

3. Algoritmos predictivos y evaluación de modelos

3.1. Algoritmos predictivos de clasificación (continuación)

3.1.1 Árbol de decisión

Es un método de aprendizaje automático utilizado para la toma de decisiones (o hacer predicciones) y la resolución de problemas de clasificación y regresión. Los árboles de decisión tienen una amplia gama de aplicaciones prácticas.

En el sector financiero, se utilizan para evaluar el riesgo de crédito de los clientes y en el *trading* para tomar decisiones de inversión basadas en patrones históricos. En el ámbito de la salud, ayudan a diagnosticar enfermedades al clasificar los síntomas y signos clínicos.

En el comercio minorista y marketing son útiles para segmentar clientes y predecir su comportamiento de compra, lo que facilita la personalización de estrategias de *marketing.*

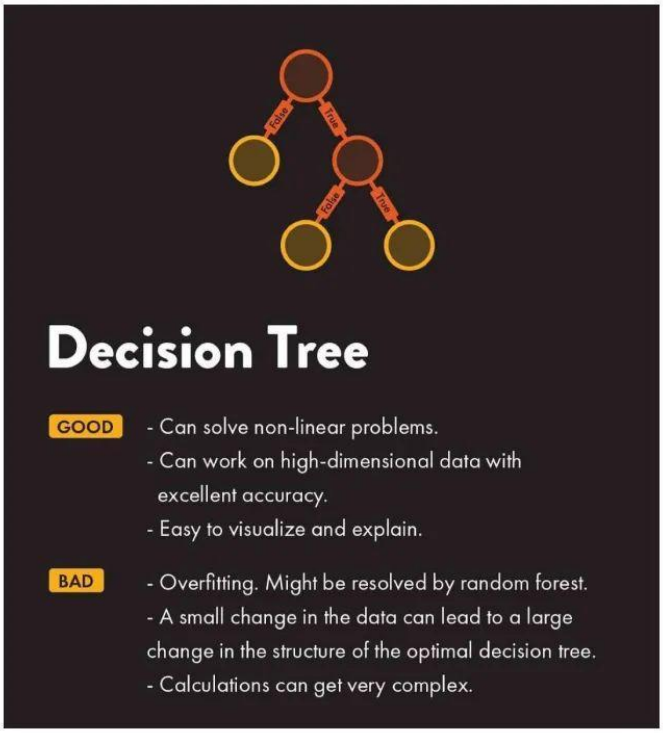
También desempeñan un papel crucial en el control de la calidad en la fabricación, identificando factores clave que influyen en los defectos del producto.

Su popularidad se debe a su facilidad de interpretación y efectividad, ya que los árboles de decisión pueden manejar tanto datos numéricos como categóricos y son capaces de modelar interacciones complejas entre variables.

En la Figura 16 indican algunos pros y contras del algoritmo de árbol de decisión.

Figura 16. Pros y contras del algoritmo de árbol de decisión

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.



Fuente: ML4Devs (2020)

3.1.2. Redes neuronales artificiales

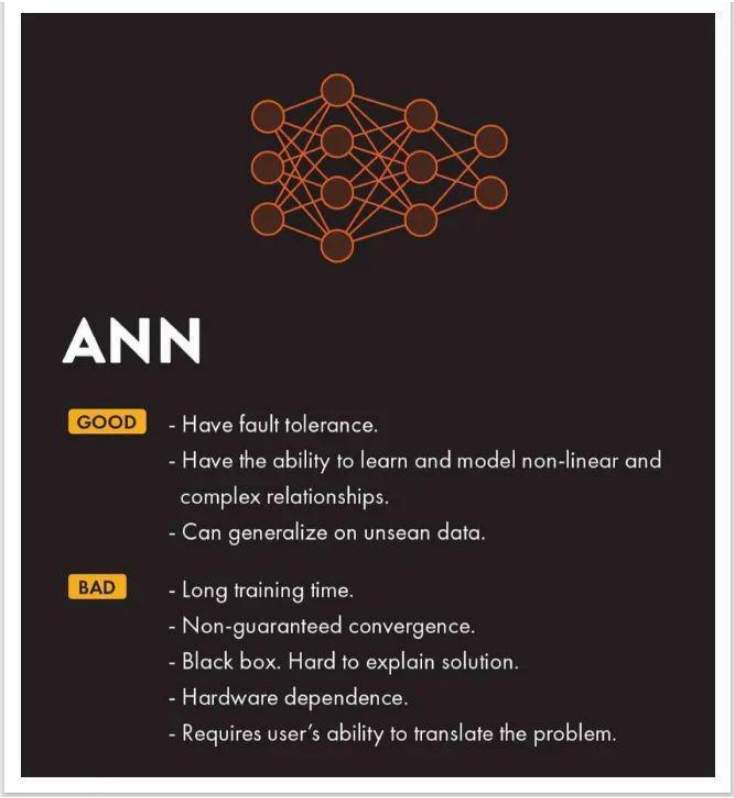
Una Red Neuronal Artificial (RNA), o artificial neural network (ANN), es un modelo de aprendizaje automático inspirado en el funcionamiento del cerebro humano. Está compuesta por unidades de procesamiento llamadas neuronas artificiales o nodos, que están organizadas en capas interconectadas. Las redes neuronales artificiales son utilizadas para abordar una amplia variedad de tareas de aprendizaje automático, incluyendo clasificación, regresión, procesamiento de imágenes, procesamiento de lenguaje natural y más. Por ejemplo, en el campo de la visión por computadora, son esenciales para el reconocimiento de imágenes y rostros, permitiendo innovaciones como los sistemas de seguridad biométrica y las aplicaciones de filtros en redes sociales. En el procesamiento del lenguaje natural, impulsan traductores automáticos y asistentes virtuales, facilitando la comunicación y la interacción con la tecnología. En el sector financiero, estas redes se utilizan para la predicción de series temporales en mercados bursátiles y para la detección de fraudes, analizando patrones complejos en transacciones. En medicina, contribuyen significativamente en el diagnóstico y pronóstico de enfermedades, analizando datos médicos y patrones en imágenes diagnósticas como radiografías y resonancias magnéticas. Además, en el transporte y ámbito

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

del vehículo autónomo, son fundamentales para la interpretación de los datos sensoriales y la toma de decisiones en tiempo real.

En la Figura 17 indican algunos pros y contras del algoritmo ANN.

Figura 17. Pros y contras del algoritmo ANN



Fuente: ML4Devs (2020)

3.2. Evaluación de modelos predictivos y métricas de rendimiento

Estimar la bondad de un clasificador sirve para medir su capacidad de predicción sobre nuevas instancias (generalización). La validación se realiza brevemente basándose en la tasa de error, entendido como la clasificación incorrecta.

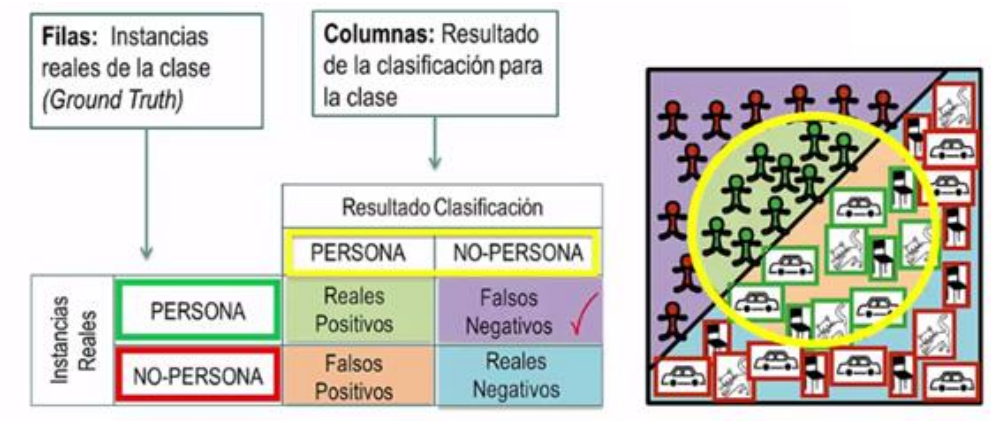
Tasa de error = # de errores cometidos / # total de casos

Una **matriz de confusión** permite ver, mediante una tabla de contingencias, la distribución de los errores (tipo I o falsos positivos y tipo II o falsos negativos)

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

cometidos por un clasificador a lo largo de las distintas categorías del problema, ver Figura 18. De una matriz de confusión se pueden extraer otros conceptos enriquecedores a la hora de comprender la distribución y naturaleza de los errores cometidos por el clasificador, así como algunas otras métricas de evaluación como: exactitud (*accuracy*), precisión (*precision*), sensibilidad (*recall*), especificidad (*specificity*), F1-score, Curva ROC, área bajo la curva (AUC), coef. Kappa, coef. de determinación R2.

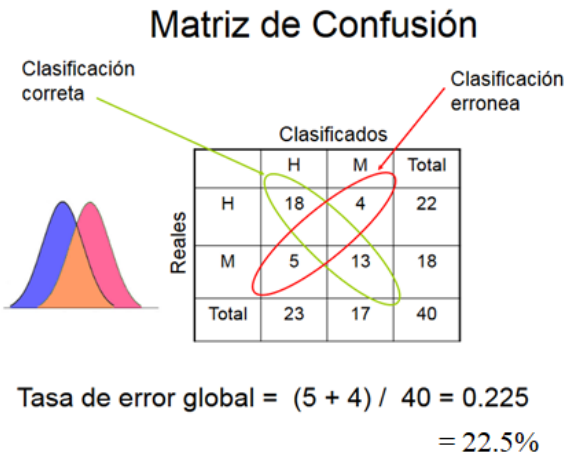
Figura 18. Matriz de confusión de un clasificador binario



Fuente: UAB (2016)

En la Figura 19 se muestra un ejemplo de un clasificador binario y la tasa de error cometido.

Figura 19. Ejemplo de matriz de confusión binaria



Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Entendiendo los términos TP, FP, FN, TN:

- Verdadero positivo (TP): el modelo predijo positivo y es cierto (asignaciones positivas correctas)
- Verdadero Negativo (TN): el modelo predijo negativo y es cierto (asignaciones negativas correctas)
- Falso positivo (FP, error tipo 1): el modelo predijo positivo y es falso (asignaciones positivas incorrectas)
- Falso negativo (FN, error tipo 2): el modelo predijo negativo y es falso (asignaciones negativas incorrectas).

En un problema de clasificación binario, se denomina como la clase positiva (P ó 1) a los ejemplos que deseamos ser capaces de identificar. Los demás ejemplos los asignaremos a la clase negativa (N ó 0). Ejemplos:

- Drowsiness detection: drowsy (1), non-drowsy (0
- Breast cancer detection: malignant (1), benign (0)
- Fraudulent transaction detector: fraud (1), non-fraud (0)
- Spam filter: spam (1), non-spam (0)

Métricas de evaluación a partir de la matriz de confusión 2×2:

- **Exactitud (*accuracy*):** calcula la tasa de aciertos totales (predicciones correctas) y da como resultado la eficacia del algoritmo.

- **Precisión (*precision*):** calcula la relación entre las predicciones positivas correctas y los pronósticos positivos totales. Indica la calidad de la respuesta del clasificador.

- **Sensibilidad (o *Recall o TPR -True Positive Rate-):*** calcula la tasa de positivos verdaderos (TP) e indica la eficiencia en la clasificación de todos los elementos que son de la clase positiva (+).

- **Especificidad:** calcula la tasa de negativos verdaderos (TN) e indica la eficiencia en la clasificación de todos los elementos que son de la clase negativa (-).

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

- **Medida F1:** conocida como media armónica de precisión y sensibilidad. Calcula el equilibrio entre la precisión y la sensibilidad, que varía de 0 a 1, siendo 1 el mejor valor.

En la Tabla 5 se presenta la matriz de confusión para la detección de somnolencia. Fíjese que la clase negativa está en la primera columna, a diferencia de la mostrada en la Figura 18. Ambas formas de organización son usadas y se deben considerar en el momento de calcular las métricas de evaluación de un algoritmo.

Tabla 5. Estructura de la Matriz de confusión para la detección de somnolencia.

		PREDICHO	
		No-Somnolencia	Somnolencia
REAL	No-Somnolencia	TN	FP
	Somnolencia	FN	TP

Las fórmulas para calcular las métricas de evaluación (a partir de la Tabla 5) son las siguientes:

$$accuracy = \frac{TP + TN}{\mathrm{TP + FN + FP + TN}}$$

$$precision = \frac{TP}{\mathrm{TP + FP}}$$

$$sensitivity\ (o\ recall) = \frac{TP}{\mathrm{TP + FN}}$$

$$specificity = \frac{TN}{\mathrm{TN + FP}}$$

$$F1\ score = \frac{2 * precisión * recall}{precisión + recall}$$

En la Tabla 6 se muestra un ejemplo de un clasificador binario para el cálculo de las métricas de evaluación.

Tabla 6. Matriz de confusión para la detección de somnolencia.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

n=165	Predicted: NO	Predicted: YES	
Actual: NO	TN = 50	FP = 10	60
Actual: YES	FN = 5	TP = 100	105
	55	110	

¿Qué se puede aprender de esta matriz de confusión?

- De esos 165 casos, el clasificador predijo "sí" 110 veces y "no" 55 veces.
- En realidad, 105 pacientes de la muestra padecen la enfermedad y 60 pacientes no padecen.

Métricas:

- Accuracy: (TP+TN)/total = (100+50)/165 = 0.91
- Error Rate: (FP+FN)/total = (10+5)/165 = 0.09
- Precision: TP/predicted yes = 100/110 = 0.91
- Sensitivity" or "Recall" (True Positive Rate): TP/actual yes = 100/105 = 0.95
- Specificity (True Negative Rate:): TN/actual no = 50/60 = 0.83
- F1-score = (2*0.91*0.95)/(0.91+0.95)= 0.93

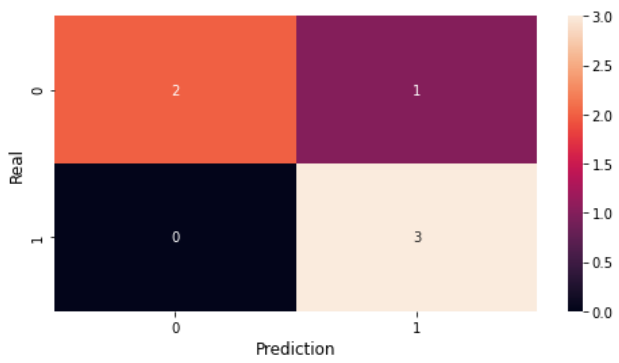
A continuación, se presenta a modo de ejemplo la evaluación de modelos en Python, la librería Scikit-learn y los resultados obtenidos (Figura 20) en un dataset pequeño conteniendo 6 observaciones:

```
# Dataset:
y_true = [0, 1, 0, 0, 1, 1]
y_pred = [0, 0, 1, 0, 0, 1]
# Matriz de confusión:
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_true, y_pred)
# Exactitud:
from sklearn.metrics import accuracy_score
accuracy_score(y_true, y_pred)
```

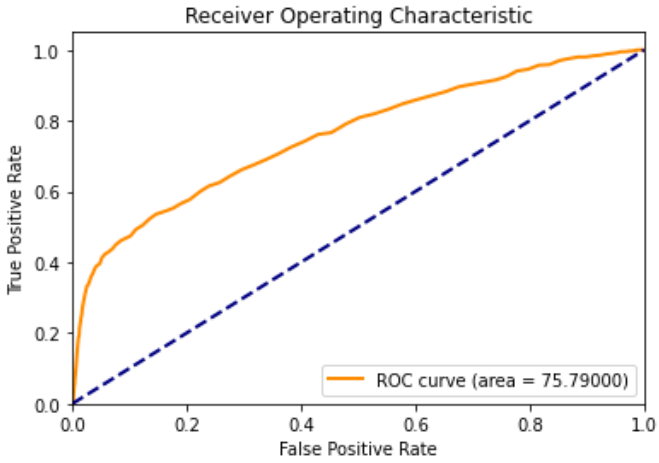
Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

 # Sensibilidad:
from sklearn.metrics import recall_score
recall_score(y_true, y_pred)
 # Especificidad:
from sklearn.metrics import recall_score
recall_score(y_true, y_pred, pos_label=0)
 # Precisión:
from sklearn.metrics import precision_score
precision_score(y_true, y_pred)
 # Puntuación F1:
from sklearn.metrics import f1_score
f1_score(y_true, y_pred)
 # Área bajo la curva:
from sklearn.metrics import roc_auc_score
auc = roc_auc_score(y_true, y_pred)
 # R Score:(R^2 coefficient of determination)
from sklearn.metrics import r2_score
r2_score(y_true, y_pred)
 # ROC Curve (binary classification problem):
from sklearn.metrics import roc_curve
import matplotlib.pyplot as plt
plt.plot(**roc_curve**(y_true, y_pred)[0], roc_curve(y_true, y_pred)[1],
color='darkorange',lw=2, label='ROC curve (area = %0.2f)' % auc)

Figura 20. Resultados del Código Python mostrando la matriz de confusión y la curva ROC.



Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.



La matriz de confusión y la curva ROC se puede adaptar para problemas de clasificación con más de 2 clases. Sin embargo, esto queda como trabajo autónomo del maestrante.

Trabajo autónomo:

Calcule la matriz de confusión y las métricas de evaluación del siguiente detector de somnolencia (2x2) dado en la Tabla 7.

Tabla 7. Valores predichos y reales para un clasificador de somnolencia

Actual class	Predicted class
1	1
0	1
0	0
1	1
1	1
0	1
0	0
0	0
1	1
1	0

		Predicted	
		Non-Drowsy (0)	Drowsy (1)
Actual	Non-Drowsy (0)		
	Drowsy (1)		

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Conclusiones

En esta unidad se ha revisado los algoritmos predictivos de clasificación: árboles de decisión y redes neuronales artificiales. Ellos tienen un sin número de aplicaciones prácticas, abarcando numerosos sectores e industrias. Sin embargo, para obtener un mejor rendimiento de cada uno de ellos es importante experimentar con los parámetros de cada algoritmo. Además, hay que considerar las ventajas y desventajas que presentan cada uno. En la Tabla 8 se indican algunos criterios a considerar para la selección adecuada de un algoritmo de clasificación de aprendizaje automático.

Tabla 8. Criterios para la selección de un algoritmo de clasificación

Algorithm	Prediction Speed	Training Speed	Memory Usage	Required Tuning	General Assessment
Logistic Regression (and Linear SVM)	Fast	Fast	Small	Minimal	Good for small problems with linear decision boundaries
Decision Trees	Fast	Fast	Small	Some	Good generalist, but prone to overfitting
(Nonlinear) SVM (and Logistic Regression)	Slow	Slow	Medium	Some	Good for many binary problems, and handles high-dimensional data well
Nearest Neighbor	Moderate	Minimal	Medium	Minimal	Lower accuracy, but easy to use and interpret
Naïve Bayes	Fast	Fast	Medium	Some	Widely used for text, including spam filtering
Ensembles	Moderate	Slow	Varies	Some	High accuracy and good performance for small- to medium-sized datasets
Neural Network	Moderate	Slow	Medium to Large	Lots	Popular for classification, compression, recognition, and forecasting

Fuente: Mathworks (2023)

Además, otra característica deseable es la **interpretabilidad** del modelo. Las ANN y SVM se consideran como cajas negras, mientras que los árboles (y la regresión logística) son mucho más interpretables (predicción □ salida).

Finalmente, se debe intentar lograr un equilibrio entre la precisión (%), el tamaño del modelo (MB) y la velocidad de inferencia (ms) del modelo entrenado, considerando en donde será puesto en producción: aplicación Web, móvil, sistema embebido.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Bibliografía:

Berzal, Fernando (2019). *Redes neuronales & Deep Learning*. USA: Independently published.

Brownlee, J. (2023). *Machine Learning Mastery*. https://machinelearningmastery.com/

García-Santillán, I. (2023). *Ciencia y analítica de datos. Maestría en Inteligencia Artificial aplicada*. Universidad Hemisferios.

InstitutoEmprende (2023). *Cómo crear un árbol de decisión*. https://institutoemprende.com/como-crear-un-arbol-de-decisiones/

Lara, J. (2014). *Minería de Datos*. Madrid: CEF-Udima

MathWorks (2023). *Machine Learning in Matlab*. https://la.mathworks.com/help/stats/machine-learning-in-matlab.html

MaximaFormacion (2023). *Qué son los árboles de decisión y para qué sirven*. https://www.maximaformacion.es/blog-dat/que-son-los-arboles-de-decision-y-para-que-sirven/

ML4Devs (2020). *An Engineer's Trek into Machine Learning*. https://www.ml4devs.com/articles/machine-learning-intro-for-developers/

Sierra, B. (2006). *Aprendizaje automático: conceptos básicos y avanzados*. Madrid: Prentice-Hall.

Torres, J. (2020). *Python Deep Learning: Introducción práctica con Keras y TensorFlow 2*. Marcombo.

UAB. (2016). *Curso de Detección de objetos*. Universidad Autónoma de Barcelona.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Evaluación de modelos de clasificación

Precisión

$$\frac{TP}{TP + FP}$$

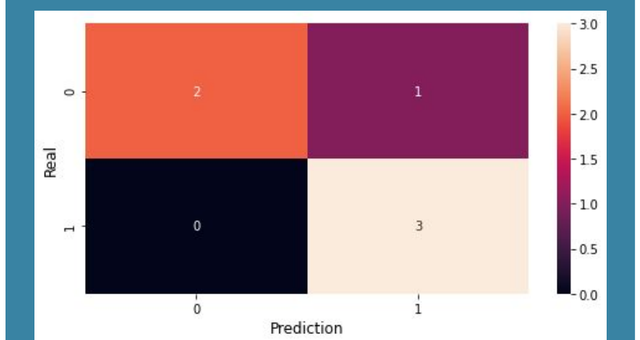
Exactitud

$$\frac{TP + TN}{TP + FN + FP + TN}$$

Especificidad

$$\frac{TN}{TN + FP}$$

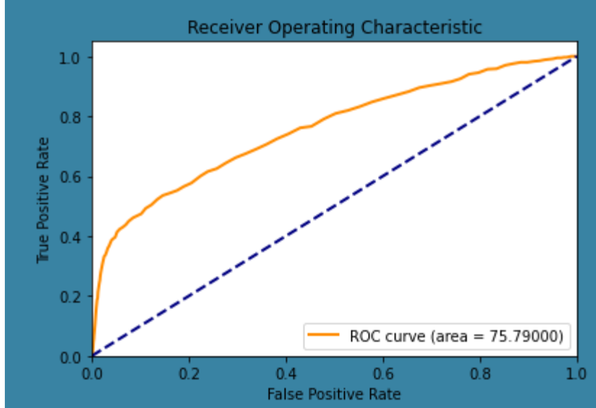
Matriz de Confusión



Sensibilidad

$$\frac{TP}{TP + FN}$$

Curva ROC



F1- Score

$$\frac{2 * precisión * recall}{precisión + recall}$$